MySQL User-Defined Functions (UDFs) — Complete Detailed Notes

Table of Contents

1. Introduction to User-Defined Functions
2. Why do we need functions?
3. Types of MySQL functions
4. Built-in functions vs User-defined functions
5. What is a stored function?
6. Complete syntax of `CREATE FUNCTION`
7. Understanding function components
8. Parameters and return types
9. `RETURNS` and `RETURN`
10. `DETERMINISTIC`, `NOT DETERMINISTIC`, and data access characteristics
11. `BEGIN ... END` and compound statements
12. Variables using `DECLARE`
13. `SET` and `SELECT ... INTO`
14. Flow control inside functions
15. Conditional logic: `IF`, `CASE`
16. Loops inside functions
17. Using functions with tables
18. Calling functions using `SELECT`

19. Functions with multiple input parameters
20. Functions that return calculated results
21. Functions that return strings, dates, and other data types
22. NULL handling
23. Error handling and validation
24. Restrictions on stored functions
25. Creating and managing functions
26. `SHOW CREATE FUNCTION` and `INFORMATION_SCHEMA`
27. `DROP FUNCTION` and `ALTER FUNCTION`
28. Functions vs stored procedures
29. Functions vs triggers
30. Practical examples with complete logic
31. Common errors and debugging
32. Performance and best practices
33. Interview questions with detailed answers
34. Practice problems with solutions
35. Quick revision notes

1. What is a User-Defined Function?

A User-Defined Function (UDF) is a function created by a programmer to perform a specific operation and return a value.

In MySQL, when we talk about ordinary SQL-level stored functions, we generally mean stored functions created using `CREATE FUNCTION`.

A stored function can:

- Accept input parameters.
- Perform calculations.
- Store intermediate values in local variables.
- Apply conditional logic.
- Execute loops.
- Read data from tables, subject to restrictions.
- Return a single value through `RETURN`.
- Be used inside SQL expressions, including `SELECT`, `WHERE`, `ORDER BY`, and `HAVING`, when the expression is otherwise valid.

1.1 Real-life example

Suppose a company calculates employee bonuses using this formula:

$$ \text{Bonus}=\frac{\text{Salary}\times\text{Bonus Rate}}{100}
$$

Without a function, you might repeatedly write the calculation:

```
SELECT
    employee_name,
    salary,
    salary * 10 / 100 AS bonus
FROM employees;
```

If the same calculation is needed in many queries, repeating the formula creates maintenance problems.

Instead, create a function:

```
DELIMITER //

CREATE FUNCTION calculate_bonus(
    p_salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_salary * 10 / 100;
END //

DELIMITER ;
```

Now call it:

```
SELECT calculate_bonus(50000.00) AS bonus;
```

Output:

bonus
5000.00

Logic: `50000 × 10 / 100 = 5000`.

The calculation is written once and can be reused in multiple queries.

> Important distinction: MySQL also supports native, compiled user-defined functions through its plugin mechanism. These are different from the SQL stored functions taught in these notes. Here, UDF refers primarily to SQL stored functions.

2. Why do we need User-Defined Functions?

Consider these common requirements:

Requirement	Function name
Calculate employee bonus	`calculate_bonus()`
Calculate simple interest	`calculate_interest()`
Convert marks into grades	`calculate_grade()`
Check even or odd	`check_even_odd()`
Calculate annual salary	`annual_salary()`
Calculate age from date of birth	`calculate_age()`
Format a person's full name	`full_name()`
Calculate a discount	`calculate_discount()`

Without functions, the same business logic may be duplicated across several queries.

With functions, the logic is centralized.

2.1 Main advantages

1. Reusability

Write the logic once and use it repeatedly.

2. Maintainability

If the business formula changes, update the function rather than every query that uses it.

3. Readability

```
SELECT calculate_bonus(salary)
FROM employees;
```

This can be easier to understand than repeating a long formula.

4. Consistency

Multiple queries can use the same implementation of a calculation.

5. Encapsulation

The calling query does not need to know every internal step of the function.

2.2 Limitations

Functions are not the right solution for every task.

- A stored function returns one value per invocation.
- It is not designed to return a normal result set containing many rows and columns.
- Stored functions have restrictions on statements they can execute.
- Calling a function separately for many rows can add execution overhead.
- A function that reads tables may produce different results as the underlying data changes.

3. Types of MySQL Functions

MySQL functions can be categorized in different ways.

3.1 Built-in functions

These are already provided by MySQL.

Examples:

```
SELECT UPPER('mysql') AS result;
```

Output: `MYSQL`

```
SELECT LENGTH('Hello') AS result;
```

Output: `5`

```
SELECT ROUND(125.678, 2) AS result;
```

Output: `125.68`

Common categories include:

- String functions: `UPPER()`, `LOWER()`, `CONCAT()`, `SUBSTRING()`.
- Numeric functions: `ABS()`, `ROUND()`, `CEIL()`, `FLOOR()`.
- Date and time functions: `CURDATE()`, `NOW()`, `DATE_ADD()`.
- Aggregate functions: `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.
- Conditional functions: `IF()`, `IFNULL()`, `COALESCE()`.
- Conversion functions: `CAST()`, `CONVERT()`.

3.2 Stored functions

These are created and stored in the database using SQL.

```
DELIMITER //

CREATE FUNCTION square_number(
    p_number INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_number * p_number;
END //

DELIMITER ;
```

Use it:

```
SELECT square_number(6) AS result;
```

Output:

result
36

3.3 Native compiled UDFs

These are implemented in compiled code and installed through MySQL's native UDF mechanism. They are not created using `CREATE FUNCTION ... BEGIN ... END`.

For ordinary database application development and your SQL learning sequence, focus on stored functions.

3.4 Scalar functions vs aggregate functions

A scalar function returns one value for each invocation.

```
SELECT UPPER('hello');
```

An aggregate function summarizes values across rows.

```
SELECT COUNT(*) FROM employees;
```

A stored function is generally scalar in its return behavior. It does not become an aggregate function merely because it reads a table.

4. Built-in Functions vs User-Defined Functions

Feature	Built-in function	Stored function
Created by	MySQL	Database developer
Available by default	Usually yes	Must be created first
Examples	`UPPER()`, `ROUND()`	`calculate_bonus()`
Custom business logic	Limited to provided behavior	Can implement custom logic
Stored in database	Built into MySQL	Yes
Called in `SELECT`	Yes, where valid	Yes, where valid
Needs `CREATE FUNCTION`	No	Yes
Can use local variables	Not applicable to caller	Yes
Can contain compound logic	Not applicable to caller	Yes

Example using both:

```
SELECT UPPER('prajwal') AS username,
       ROUND(125.678, 2) AS rounded_value;
```

A stored function can also use built-in functions internally.

```
DELIMITER //

CREATE FUNCTION format_name(
    p_name VARCHAR(100)
)
RETURNS VARCHAR(100)
DETERMINISTIC
NO SQL
BEGIN
    RETURN UPPER(TRIM(p_name));
END //

DELIMITER ;
```

Call:

```
SELECT format_name('  prajwal  ') AS formatted_name;
```

Output:

formatted_name
PRAJWAL

Logic: `TRIM()` removes surrounding spaces; `UPPER()` converts the remaining text to uppercase.

5. Stored Function: Complete Basic Syntax

The general syntax is:

```
DELIMITER //

CREATE FUNCTION function_name(
    parameter_name parameter_data_type,
    parameter_name parameter_data_type
)
RETURNS return_data_type
[characteristics]
BEGIN
    -- Declare local variables first, if required.
    -- Write SQL statements and calculations.
    RETURN expression;
END //

DELIMITER ;
```

The square brackets above indicate optional syntax elements. They are not typed literally.

A function may also have a simple single-statement body without `BEGIN ... END` when appropriate.

Example:

```
DELIMITER //

CREATE FUNCTION add_numbers(
    p_a INT,
    p_b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
RETURN p_a + p_b //

DELIMITER ;
```

Call:

```
SELECT add_numbers(10, 20) AS result;
```

Output: `30`

The compound form is useful when the function needs multiple statements, variables, or conditional logic.

6. Understanding Every Component of `CREATE FUNCTION`

Consider:

```
DELIMITER //

CREATE FUNCTION calculate_bonus(
    p_salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_bonus DECIMAL(10,2);

    SET v_bonus = p_salary * 10 / 100;

    RETURN v_bonus;
END //

DELIMITER ;
```

6.1 `DELIMITER //`

The default MySQL client statement delimiter is usually `;`.

However, the function body itself contains semicolons:

```
DECLARE v_bonus DECIMAL(10,2);

SET v_bonus = p_salary * 10 / 100;

RETURN v_bonus;
```

If the client treated every semicolon as the end of the entire `CREATE FUNCTION` statement, it would split the definition incorrectly.

So, in clients that support the `DELIMITER` command, we temporarily change the delimiter.

```
DELIMITER //
```

Now `//` marks the end of the complete function definition.

After creation:

```
DELIMITER ;
```

This restores the usual delimiter.

Important: `DELIMITER` is a client command, not a SQL statement executed by the MySQL server. Some GUI tools and APIs manage statement boundaries differently; in those environments, follow the tool's rules.

6.2 `CREATE FUNCTION`

This tells MySQL to create a stored function.

```
CREATE FUNCTION calculate_bonus(...)
```

6.3 Function name

```
calculate_bonus
```

The name identifies the function when calling it.

Use descriptive names that communicate the purpose.

Examples:

- `calculate_bonus`
- `get_full_name`
- `calculate_discount`
- `is_eligible_for_discount`

Avoid names that conflict with existing built-in functions or create ambiguity.

6.4 Parameters

```
p_salary DECIMAL(10,2)
```

The parameter receives an input value.

Here:

- `p_salary` is the parameter name.
- `DECIMAL(10,2)` is its data type.
- `p_` is a naming convention meaning parameter; it is not a MySQL requirement.

For stored functions, parameters are input parameters. Unlike stored procedures, functions do not declare `OUT` or `INOUT` parameters.

6.5 `RETURNS`

```
RETURNS DECIMAL(10,2)
```

This specifies the function's return data type.

The function is expected to return a value compatible with that type.

Examples:

```
RETURNS INT
```

```
RETURNS DECIMAL(10,2)
```

```
RETURNS VARCHAR(100)
```

```
RETURNS DATE
```

```
RETURNS DATETIME
```

6.6 Characteristics

```
DETERMINISTIC
NO SQL
```

These declare properties of the function. They are discussed in detail below.

6.7 `BEGIN ... END`

Defines a compound statement containing multiple statements.

```
BEGIN
    DECLARE v_bonus DECIMAL(10,2);
    SET v_bonus = p_salary * 10 / 100;
    RETURN v_bonus;
END
```

6.8 `DECLARE`

Creates a local variable.

```
DECLARE v_bonus DECIMAL(10,2);
```

This variable exists within the function's relevant block and stores an intermediate result.

6.9 `SET`

Assigns a value to a variable.

```
SET v_bonus = p_salary * 10 / 100;
```

6.10 `RETURN`

Returns the result to the caller.

```
RETURN v_bonus;
```

This is one of the most important statements in a stored function.

7. Parameters and Return Types

7.1 One parameter

```
DELIMITER //

CREATE FUNCTION double_number(
    p_number INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_number * 2;
END //

DELIMITER ;
```

Call:

```
SELECT double_number(8);
```

Output: `16`

7.2 Multiple parameters

```
DELIMITER //

CREATE FUNCTION multiply_numbers(
    p_a INT,
    p_b INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_a * p_b;
END //

DELIMITER ;
```

Call:

```
SELECT multiply_numbers(5, 4);
```

Output: `20`

Parameters are matched by position when the function is called.

```
SELECT multiply_numbers(5, 4);
```

Here, `p_a = 5` and `p_b = 4`.

7.3 Common return types

Requirement	Return type example
Whole-number calculation	`INT`
Large integer calculation	`BIGINT`

Requirement	Return type example
Money or precise decimal	`DECIMAL(10,2)`
Text	`VARCHAR(100)`
Single date	`DATE`
Date and time	`DATETIME`
True/false-style result	`BOOLEAN` or `TINYINT`

MySQL's `BOOLEAN` is an alias for `TINYINT(1)`, not a separate Boolean storage type.

For monetary calculations, `DECIMAL` is generally preferable to floating-point types when exact decimal arithmetic is required.

Return-type precision

Choose a return type that can represent the expected result. A function declared with `RETURNS INT` should not be expected to preserve arbitrary decimal precision.

For example:

```
SELECT CAST(12.75 AS SIGNED) AS result;
```

This illustrates integer conversion; the decimal fraction is not preserved.

For money, use a suitable decimal return type instead.

8. `RETURNS` vs `RETURN`

These two keywords have different purposes.

8.1 `RETURNS`

Declares the return type in the function definition.

```
RETURNS DECIMAL(10,2)
```

8.2 `RETURN`

Returns the actual value during execution.

```
RETURN p_salary * 0.10;
```

Example:

```
DELIMITER //

CREATE FUNCTION tax_amount(
    p_salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_salary * 0.05;
END //

DELIMITER ;
```

Call:

```
SELECT tax_amount(40000.00);
```

Output: `2000.00`

Remember:

- `RETURNS` answers: What type of value will the function return?
- `RETURN` answers: What value should this invocation return?

A stored function must have a valid return path. A function with no valid executed `RETURN` may produce `NULL` in certain execution paths, while invalid definitions or other errors can arise depending on the exact code and SQL mode. Design every path to return an intentional value.

9. `DETERMINISTIC`, `NOT DETERMINISTIC`, `NO SQL`, and Data Access

Function characteristics describe properties of a stored function.

They are especially important for database administration, binary logging, replication, and correctness.

9.1 `DETERMINISTIC`

A deterministic function returns the same result for the same inputs, assuming the same relevant state and execution conditions.

Example:

```
DELIMITER //

CREATE FUNCTION square_value(
    p_number INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_number * p_number;
END //

DELIMITER ;
```

```
SELECT square_value(5);
```

Output: `25`

Repeated calls with the same input return the same result.

9.2 `NOT DETERMINISTIC`

A function is not deterministic when the same inputs can produce different outputs.

For example, a function that uses the current time:

```
DELIMITER //

CREATE FUNCTION current_timestamp_value()
RETURNS DATETIME
NOT DETERMINISTIC
NO SQL
BEGIN
    RETURN NOW();
END //

DELIMITER ;
```

Calling it at different times may produce different results.

```
SELECT current_timestamp_value();
```

The result depends on the time of execution.

9.3 Data access characteristics

MySQL supports these data access characteristics:

Characteristic	Meaning
`NO SQL`	The function contains no SQL statements that read or modify data.
`CONTAINS SQL`	The function contains SQL statements but does not declare that it reads or modifies data.
`READS SQL DATA`	The function reads data but does not modify it.
`MODIFIES SQL DATA`	The function modifies data.

These characteristics are declarations; they are not a complete runtime permission system and do not override stored-function restrictions.

Example: `NO SQL`

```
DELIMITER //

CREATE FUNCTION add_values(
    p_a INT,
    p_b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_a + p_b;
END //

DELIMITER ;
```

Example: `READS SQL DATA`

```
DELIMITER //

CREATE FUNCTION get_employee_count()
RETURNS BIGINT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count BIGINT;

    SELECT COUNT(*)
    INTO v_count
    FROM employees;

    RETURN v_count;
END //

DELIMITER ;
```

This function reads the `employees` table.

Its result can change when rows are inserted or deleted. Declaring it `NOT DETERMINISTIC` is appropriate when it is intended to reflect current table state.

Important rule

Do not label a function `DETERMINISTIC` merely to make it look optimized or to bypass a database configuration requirement.

Declare characteristics truthfully. In MySQL, some characteristics are used by the server as declarations rather than verified proofs of behavior.

Binary logging note

Depending on MySQL version, server configuration, privileges, and binary logging settings, creating or executing stored functions can be subject to additional

restrictions. A function's characteristics may affect whether the server permits its creation or execution under those settings.

10. `BEGIN ... END` and Compound Statements

Use `BEGIN ... END` when your function requires multiple statements.

Example:

```
DELIMITER //

CREATE FUNCTION calculate_total(
    p_price DECIMAL(10,2),
    p_quantity INT
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_total DECIMAL(12,2);

    SET v_total = p_price * p_quantity;

    RETURN v_total;
END //

DELIMITER ;
```

Call:

```
SELECT calculate_total(250.00, 4) AS total;
```

Output: `1000.00`

Execution flow

1. The caller passes `250.00` and `4`.

2. MySQL assigns the values to `p_price` and `p_quantity`.
3. The function declares `v_total`.
4. `SET` calculates `250 × 4`.
5. `RETURN` sends `1000.00` to the caller.
6. The caller displays the result.

Rule: declaration order

Inside a compound block, local declarations must appear before executable statements.

Correct:

```
BEGIN
    DECLARE v_total DECIMAL(12,2);
    DECLARE v_tax DECIMAL(12,2);

    SET v_total = 1000.00;
    SET v_tax = v_total * 0.05;

    RETURN v_tax;
END
```

Incorrect:

```
BEGIN
    SET v_total = 1000.00;
    DECLARE v_total DECIMAL(12,2);
    RETURN v_total;
END
```

The declaration appears after an executable statement and is invalid.

When using cursors and condition handlers, MySQL has additional declaration-order rules: variables and conditions first, then cursors, then handlers.

11. Local Variables: `DECLARE`, `SET`, and `SELECT ... INTO`

Local variables store intermediate results.

11.1 `DECLARE`

Syntax:

```
DECLARE variable_name data_type [DEFAULT default_value];
```

Example:

```
DECLARE v_salary DECIMAL(10,2);
```

With a default:

```
DECLARE v_bonus DECIMAL(10,2) DEFAULT 0.00;
```

Local variables without an explicit default begin with `NULL`.

11.2 `SET`

Syntax:

```
SET variable_name = expression;
```

Example:

```
SET v_bonus = p_salary * 0.10;
```

Multiple assignments can also be written using `SET`:

```
SET v_a = 10, v_b = 20;
```

11.3 `SELECT ... INTO`

Use `SELECT ... INTO` to put a query result into local variables.

Example:

```
DECLARE v_total BIGINT;

SELECT COUNT(*)
INTO v_total
FROM employees;
```

The query returns one count, which is assigned to `v_total`.

`SET` vs `SELECT ... INTO`

`SET`	`SELECT ... INTO`
Assigns an expression	Assigns query result columns
Useful for calculations	Useful for reading table data
Example: `SET v_x = 10;`	Example: `SELECT COUNT(*) INTO v_x FROM employees;`

Important `SELECT ... INTO` rules

When assigning values from a query:

- The query should return the expected number of columns.
- The selected column count must match the number of target variables.
- A query returning more than one row can raise an error.
- If no row is returned, variables can remain unchanged and a `NOT FOUND` condition may occur.
- Use aggregates such as `COUNT(*)` when you need a guaranteed single-row result, remembering that the aggregate value itself may be `0`.

12. Conditional Logic Inside Functions

Stored functions support compound statements and MySQL flow-control constructs, subject to the rules of stored programs.

The main constructs include:

- `IF ... THEN ... ELSEIF ... ELSE ... END IF`
- `CASE ... END CASE`
- `WHILE ... DO ... END WHILE`
- `REPEAT ... UNTIL ... END REPEAT`
- `LOOP ... END LOOP`
- `LEAVE`
- `ITERATE`

Use conditionals when the result depends on an input or a calculated value.

12.1 `IF ... ELSE`

Syntax:

```
IF condition THEN
    statements;
ELSE
    statements;
END IF;
```

Example: even or odd.

```
DELIMITER //

CREATE FUNCTION even_or_odd(
    p_number INT
)
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
    IF MOD(p_number, 2) = 0 THEN
        RETURN 'Even';
    ELSE
        RETURN 'Odd';
    END IF;
END //

DELIMITER ;
```

Call:

```
SELECT even_or_odd(25) AS result;
SELECT even_or_odd(40) AS result;
```

Output:

result
Odd
Even

Logic: `MOD(number, 2)` gives the remainder after division by 2. A remainder of zero means even.

12.2 `IF ... ELSEIF ... ELSE`

Example: grade calculation.

```
DELIMITER //

CREATE FUNCTION grade_from_marks(
    p_marks DECIMAL(5,2)
)
RETURNS VARCHAR(2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_marks IS NULL OR p_marks < 0 OR p_marks > 100 THEN
        RETURN 'NA';
    ELSEIF p_marks >= 90 THEN
        RETURN 'A+';
    ELSEIF p_marks >= 80 THEN
        RETURN 'A';
    ELSEIF p_marks >= 70 THEN
        RETURN 'B';
    ELSEIF p_marks >= 60 THEN
        RETURN 'C';
    ELSEIF p_marks >= 50 THEN
        RETURN 'D';
    ELSE
        RETURN 'F';
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT grade_from_marks(95) AS grade;
SELECT grade_from_marks(82) AS grade;
SELECT grade_from_marks(45) AS grade;
SELECT grade_from_marks(NULL) AS grade;
```

Expected output:

Input	Grade
95	A+
82	A
45	F
`NULL`	NA

Why check the range first?

If the function checked only `p_marks >= 90`, an invalid value such as `150` would incorrectly receive `A+`.

Validation should occur before normal grading.

12.3 `CASE`

`CASE` is useful when multiple alternatives exist.

Example:

```
DELIMITER //

CREATE FUNCTION day_type(
    p_day_number INT
)
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
    CASE p_day_number
        WHEN 1 THEN RETURN 'Monday';
        WHEN 2 THEN RETURN 'Tuesday';
        WHEN 3 THEN RETURN 'Wednesday';
        WHEN 4 THEN RETURN 'Thursday';
        WHEN 5 THEN RETURN 'Friday';
        WHEN 6 THEN RETURN 'Saturday';
        WHEN 7 THEN RETURN 'Sunday';
        ELSE RETURN 'Invalid';
    END CASE;
END //

DELIMITER ;
```

Test:

```
SELECT day_type(1);
SELECT day_type(7);
SELECT day_type(10);
```

Output: `Monday`, `Sunday`, `Invalid`.

The example uses a custom mapping where 1 means Monday. This mapping is not a universal rule for all MySQL date functions.

Simple `CASE` vs searched `CASE`

Simple `CASE` compares one expression against values:

```
CASE p_status
    WHEN 1 THEN ...
    WHEN 2 THEN ...
    ELSE ...
END CASE;
```

Searched `CASE` evaluates conditions:

```
CASE
    WHEN p_marks >= 90 THEN ...
    WHEN p_marks >= 75 THEN ...
    ELSE ...
END CASE;
```

For functions, distinguish the stored-program `CASE` statement from the SQL `CASE` expression. The statement uses `END CASE`; the expression uses `END`.

13. Loops Inside Functions

Loops are useful for repetitive operations, but many numerical calculations can be expressed more simply using built-in functions or formulas.

13.1 `WHILE` loop

Syntax:

```
WHILE condition DO
    statements;
END WHILE;
```

Example: sum numbers from 1 to `n`.

```
DELIMITER //

CREATE FUNCTION sum_to_n(
    p_n INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_i INT DEFAULT 1;
    DECLARE v_sum BIGINT DEFAULT 0;

    WHILE v_i <= p_n DO
        SET v_sum = v_sum + v_i;
        SET v_i = v_i + 1;
    END WHILE;

    RETURN v_sum;
END //

DELIMITER ;
```

Call:

```
SELECT sum_to_n(5) AS total;
```

Output:

total
15

Dry run

Iteration	`v_i` before addition	`v_sum` after addition
1	1	1

Iteration	`v_i` before addition	`v_sum` after addition
2	2	3
3	3	6
4	4	10
5	5	15

After the fifth iteration, `v_i` becomes `6`. The condition `6 <= 5` is false, so the loop ends.

13.2 `REPEAT ... UNTIL`

Syntax:

```
REPEAT
    statements;
UNTIL condition
END REPEAT;
```

The condition is checked after executing the body. Therefore, the body runs at least once.

Example:

```
DELIMITER //

CREATE FUNCTION sum_to_n_repeat(
    p_n INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_i INT DEFAULT 1;
    DECLARE v_sum BIGINT DEFAULT 0;

    IF p_n <= 0 THEN
        RETURN 0;
    END IF;

    REPEAT
        SET v_sum = v_sum + v_i;
        SET v_i = v_i + 1;
    UNTIL v_i > p_n
    END REPEAT;

    RETURN v_sum;
END //

DELIMITER ;
```

Call:

```
SELECT sum_to_n_repeat(5);
```

Output: `15`

The initial validation avoids running the body for zero or negative inputs.

13.3 `LOOP`, `LEAVE`, and `ITERATE`

`LOOP` repeats until explicitly exited.

- `LEAVE` exits a labeled block or loop.
- `ITERATE` skips to the next iteration of a labeled loop.

Example:

```
DELIMITER //

CREATE FUNCTION sum_with_loop(
    p_n INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_i INT DEFAULT 1;
    DECLARE v_sum BIGINT DEFAULT 0;

    my_loop: LOOP
        IF v_i > p_n THEN
            LEAVE my_loop;
        END IF;

        SET v_sum = v_sum + v_i;
        SET v_i = v_i + 1;
    END LOOP my_loop;

    RETURN v_sum;
END //

DELIMITER ;
```

Test:

```
SELECT sum_with_loop(5);
```

Output: `15`

Loop safety

Always ensure that a loop has a termination condition. An incorrect loop can consume excessive resources or run indefinitely.

For example, forgetting to increment `v_i` in `sum_to_n()` can cause an infinite loop when `p_n >= 1`.

14. Calling a Stored Function

A stored function is normally invoked as part of an expression.

14.1 Using `SELECT`

```
SELECT square_value(5) AS answer;
```

Output: `25`

14.2 Passing literals

```
SELECT calculate_total(100.00, 3) AS total;
```

Output: `300.00`

14.3 Passing table columns

Assume the `employees` table has `employee_id`, `employee_name`, and `salary`.

```
SELECT
    employee_id,
    employee_name,
    salary,
    calculate_bonus(salary) AS bonus
FROM employees;
```

The function is invoked with each row's `salary` value.

If there are 100 qualifying rows, the function expression may be evaluated for those rows as required by the execution plan. Do not assume that MySQL automatically caches a result for repeated inputs.

14.4 Using a function in `WHERE`

```
SELECT employee_name, salary
FROM employees
WHERE calculate_bonus(salary) > 5000;
```

This returns employees whose calculated bonus exceeds 5000.

Performance note: Applying a function to a column in a filtering condition can prevent an ordinary index on that column from being used efficiently, depending on the expression and execution plan. Consider an equivalent direct comparison when possible.

14.5 Using a function in `ORDER BY`

```
SELECT employee_name, salary
FROM employees
ORDER BY calculate_bonus(salary) DESC;
```

Employees are ordered by calculated bonus, highest first.

14.6 Using a function in `HAVING`

```
SELECT
    department_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING calculate_bonus(AVG(salary)) > 5000;
```

This is valid in principle if the function accepts the resulting expression type and the grouped query is otherwise valid.

However, a function that reads tables or depends on external state should be used carefully in grouped expressions because its behavior may not correspond to a simple aggregation over the rows.

15. Creating a Complete Practice Database

The following examples use an employee table so that you can practise calculations, validation, and functions that read table data.

```
CREATE DATABASE IF NOT EXISTS function_practice;

USE function_practice;

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary DECIMAL(10,2) NOT NULL,
    experience_years INT,
    joining_date DATE
);

INSERT INTO employees
    (employee_id, employee_name, department, salary, experience_years,
joining_date)
VALUES
    (1, 'Amit', 'IT', 50000.00, 3, '2022-01-10'),
    (2, 'Priya', 'HR', 40000.00, 5, '2020-03-15'),
    (3, 'Rahul', 'IT', 65000.00, 7, '2018-07-20'),
    (4, 'Sneha', 'Finance', 55000.00, 4, '2021-06-01'),
    (5, 'Vikas', 'IT', 30000.00, 1, '2025-02-10');
```

Verify:

```
SELECT * FROM employees;
```

Expected data:

employee_id	employee_name	department	salary	experience_years	joining_date
1	Amit	IT	50000.00	3	2022-01-10
2	Priya	HR	40000.00	5	2020-03-15

employee_id	employee_name	department	salary	experience_years	joining_date
3	Rahul	IT	65000.00	7	2018-07-20
4	Sneha	Finance	55000.00	4	2021-06-01
5	Vikas	IT	30000.00	1	2025-02-10

16. Practical Function Examples

Example 1: Calculate annual salary

Assume monthly salary is multiplied by 12.

```
DELIMITER //

CREATE FUNCTION annual_salary(
    p_monthly_salary DECIMAL(10,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_monthly_salary * 12;
END //

DELIMITER ;
```

Test:

```
SELECT annual_salary(50000.00) AS annual_salary;
```

Output: `600000.00`

Use it with the table:

```
SELECT
    employee_name,
    salary AS monthly_salary,
    annual_salary(salary) AS annual_salary
FROM employees;
```

Expected result:

employee_name	monthly_salary	annual_salary
Amit	50000.00	600000.00
Priya	40000.00	480000.00
Rahul	65000.00	780000.00
Sneha	55000.00	660000.00
Vikas	30000.00	360000.00

Example 2: Calculate bonus based on salary

Business rule:

- Salary below 40,000: 5% bonus.
- Salary from 40,000 to below 60,000: 10% bonus.
- Salary of 60,000 or more: 15% bonus.

```
DELIMITER //

CREATE FUNCTION salary_bonus(
    p_salary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_salary IS NULL OR p_salary < 0 THEN
        RETURN NULL;
    ELSEIF p_salary < 40000 THEN
        RETURN p_salary * 0.05;
    ELSEIF p_salary < 60000 THEN
        RETURN p_salary * 0.10;
    ELSE
        RETURN p_salary * 0.15;
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT salary_bonus(30000.00) AS bonus;
SELECT salary_bonus(50000.00) AS bonus;
SELECT salary_bonus(65000.00) AS bonus;
```

Output: `1500.00`, `5000.00`, `9750.00`.

Apply it to employees:

```
SELECT
    employee_name,
    salary,
    salary_bonus(salary) AS bonus
FROM employees;
```

Expected result:

employee_name	salary	bonus
Amit	50000.00	5000.00
Priya	40000.00	4000.00
Rahul	65000.00	9750.00
Sneha	55000.00	5500.00
Vikas	30000.00	1500.00

Example 3: Calculate simple interest

Formula:

$$ SI=\frac{P\times R\times T}{100}
$$

Where:

- \(P\) = principal.
- \(R\) = annual interest rate in percent.
- \(T\) = time in years.

```
DELIMITER //

CREATE FUNCTION calculate_interest(
    p_principal DECIMAL(12,2),
    p_rate DECIMAL(6,3),
    p_time DECIMAL(6,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_principal IS NULL
       OR p_rate IS NULL
       OR p_time IS NULL
       OR p_principal < 0
       OR p_rate < 0
       OR p_time < 0 THEN
        RETURN NULL;
    END IF;

    RETURN p_principal * p_rate * p_time / 100;
END //

DELIMITER ;
```

Test:

```
SELECT calculate_interest(10000, 5, 2) AS simple_interest;
```

Output: `1000.00`

Logic:

$$ \frac{10000\times5\times2}{100}=1000
$$

This example returns `NULL` for invalid or missing input instead of silently treating invalid values as zero.

Example 4: Calculate discount

Business rule: discount is 10% if the amount is at least 5,000; otherwise, no discount.

```
DELIMITER //

CREATE FUNCTION calculate_discount(
    p_amount DECIMAL(12,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_amount IS NULL OR p_amount < 0 THEN
        RETURN NULL;
    ELSEIF p_amount >= 5000 THEN
        RETURN p_amount * 0.10;
    ELSE
        RETURN 0.00;
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT calculate_discount(6000.00) AS discount;
SELECT calculate_discount(3000.00) AS discount;
```

Output: `600.00` and `0.00`.

Example 5: Return full name

```
DELIMITER //

CREATE FUNCTION get_full_name(
    p_first_name VARCHAR(50),
    p_last_name VARCHAR(50)
)
RETURNS VARCHAR(101)
DETERMINISTIC
NO SQL
BEGIN
    RETURN CONCAT(
        COALESCE(TRIM(p_first_name), ''),
        ' ',
        COALESCE(TRIM(p_last_name), '')
    );
END //

DELIMITER ;
```

Test:

```
SELECT get_full_name('Amit', 'Sharma') AS full_name;
```

Output: `Amit Sharma`

Logic: `TRIM()` removes surrounding spaces. `COALESCE()` substitutes an empty string when an input is `NULL`. `CONCAT()` joins the strings.

The function above may produce a leading or trailing space if one name is missing. If you need cleaner formatting, use:

```
DELIMITER //

CREATE FUNCTION clean_full_name(
    p_first_name VARCHAR(50),
    p_last_name VARCHAR(50)
)
RETURNS VARCHAR(101)
DETERMINISTIC
NO SQL
BEGIN
    RETURN NULLIF(
        TRIM(
            CONCAT_WS(
                ' ',
                NULLIF(TRIM(p_first_name), ''),
                NULLIF(TRIM(p_last_name), '')
            )
        ),
        ''
    );
END //

DELIMITER ;
```

`CONCAT_WS()` skips `NULL` arguments, and `NULLIF()` converts empty strings into `NULL`. The result is `NULL` when both names are missing or blank.

Example 6: Get employee count from a table

```
DELIMITER //

CREATE FUNCTION total_employees()
RETURNS BIGINT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count BIGINT;

    SELECT COUNT(*)
    INTO v_count
    FROM employees;

    RETURN v_count;
END //

DELIMITER ;
```

Call:

```
SELECT total_employees() AS total;
```

Output: `5`

This function reads the current table data.

Example 7: Get the salary of an employee

```
DELIMITER //

CREATE FUNCTION get_employee_salary(
    p_employee_id INT
)
RETURNS DECIMAL(10,2)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_salary DECIMAL(10,2);

    SELECT salary
    INTO v_salary
    FROM employees
    WHERE employee_id = p_employee_id;

    RETURN v_salary;
END //

DELIMITER ;
```

Test:

```
SELECT get_employee_salary(3) AS salary;
```

Output: `65000.00`

What if the employee does not exist?

A `SELECT ... INTO` that returns no rows raises a `NOT FOUND` condition. Without appropriate handling, the function may terminate with an error instead of returning `NULL`.

A more robust version handles that condition:

```
DELIMITER //

CREATE FUNCTION safe_employee_salary(
    p_employee_id INT
)
RETURNS DECIMAL(10,2)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_salary DECIMAL(10,2) DEFAULT NULL;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_salary = NULL;

    SELECT salary
    INTO v_salary
    FROM employees
    WHERE employee_id = p_employee_id;

    RETURN v_salary;
END //

DELIMITER ;
```

Test:

```
SELECT safe_employee_salary(3) AS salary;
SELECT safe_employee_salary(999) AS salary;
```

Expected results:

Input	salary
3	65000.00
999	`NULL`

This handler is scoped to the function block. For more complex functions, think carefully about which statement might raise `NOT FOUND` and how the handler should respond.

17. NULL Handling in Functions

`NULL` means missing or unknown data. It is not the same as zero or an empty string.

17.1 Arithmetic with `NULL`

```
SELECT 100 + NULL AS result;
```

Result: `NULL`.

17.2 Comparison with `NULL`

Do not write:

```
SELECT NULL = NULL;
```

expecting `TRUE`. In SQL, this comparison evaluates to `NULL` (unknown).

Use:

```
SELECT NULL IS NULL;
```

Output: `1` (true).

17.3 Handling a NULL parameter

```
DELIMITER //

CREATE FUNCTION safe_double(
    p_number INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    IF p_number IS NULL THEN
        RETURN 0;
    END IF;

    RETURN p_number * 2;
END //

DELIMITER ;
```

Call:

```
SELECT safe_double(NULL);
```

Output: `0`

Here, the business rule explicitly says that a missing number should return zero.

That is not a universal rule. If missing input is invalid, returning `NULL` or raising an error may be more appropriate.

17.4 `COALESCE()`

`COALESCE()` returns the first non-`NULL` argument.

```
SELECT COALESCE(NULL, NULL, 50, 100) AS result;
```

Output: `50`.

Inside a function:

```
RETURN COALESCE(p_amount, 0);
```

This substitutes zero for a `NULL` amount.

17.5 Important three-valued logic rule

SQL conditions can evaluate to `TRUE`, `FALSE`, or `UNKNOWN`.

For example:

```
IF p_marks >= 50 THEN
    RETURN 'Pass';
ELSE
    RETURN 'Fail';
END IF;
```

If `p_marks` is `NULL`, the condition is unknown, so the `ELSE` branch runs.

If `NULL` must be treated differently, check it explicitly:

```
IF p_marks IS NULL THEN
    RETURN 'Missing';
ELSEIF p_marks >= 50 THEN
    RETURN 'Pass';
ELSE
    RETURN 'Fail';
END IF;
```

18. Error Handling and Validation

A function should have a defined response to invalid input.

Common validation cases include:

- Missing inputs.
- Negative amounts where negatives are not allowed.
- Marks outside the permitted range.
- Zero or negative divisors.
- Missing rows in table lookups.
- Unexpected SQL conditions.

18.1 Returning `NULL`

For a calculation that cannot be performed:

```
DELIMITER //

CREATE FUNCTION safe_divide(
    p_numerator DECIMAL(12,4),
    p_denominator DECIMAL(12,4)
)
RETURNS DECIMAL(16,4)
DETERMINISTIC
NO SQL
BEGIN
    IF p_numerator IS NULL
       OR p_denominator IS NULL
       OR p_denominator = 0 THEN
        RETURN NULL;
    END IF;

    RETURN p_numerator / p_denominator;
END //

DELIMITER ;
```

Test:

```
SELECT safe_divide(10, 2);
SELECT safe_divide(10, 0);
```

Results: `5.0000` and `NULL`.

18.2 Raising an error with `SIGNAL`

If invalid input should be rejected rather than converted into `NULL`, use `SIGNAL SQLSTATE`.

```
DELIMITER //

CREATE FUNCTION validated_divide(
    p_numerator DECIMAL(12,4),
    p_denominator DECIMAL(12,4)
)
RETURNS DECIMAL(16,4)
DETERMINISTIC
NO SQL
BEGIN
    IF p_numerator IS NULL OR p_denominator IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Inputs cannot be NULL';
    END IF;

    IF p_denominator = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Division by zero is not allowed';
    END IF;

    RETURN p_numerator / p_denominator;
END //

DELIMITER ;
```

Test:

```
SELECT validated_divide(10, 2);
```

Returns `5.0000`.

```
SELECT validated_divide(10, 0);
```

Raises an error with the custom message.

Returning NULL vs raising an error

Situation	Possible strategy
Missing optional value	Return `NULL`
Missing employee record	Return `NULL` if that is the documented behavior
Invalid marks	Return a special status or raise an error
Division by zero	Return `NULL` or raise an error
Invalid financial input	Often reject explicitly

Choose the behavior according to the application requirement. Do not use an arbitrary sentinel such as `-1` for every error; it may be confused with a valid result.

19. Restrictions on MySQL Stored Functions

Stored functions have important limitations. These rules distinguish them from procedures.

19.1 No result-set-producing statements

A stored function cannot use a normal `SELECT` that sends a result set directly to the client.

Not appropriate inside a function:

```
SELECT * FROM employees;
```

To retrieve data into variables, use:

```
SELECT salary
INTO v_salary
FROM employees
WHERE employee_id = p_employee_id;
```

This form assigns the result rather than returning a result set to the caller.

19.2 No explicit transaction control

Stored functions cannot use transaction-control statements such as:

```
START TRANSACTION;
COMMIT;
ROLLBACK;
```

Transaction management should be performed by the calling application or an appropriate procedure/workflow.

19.3 No stored procedure calls that return result sets

Stored functions cannot call stored procedures that return result sets to the client. More generally, stored-program calls must respect the restrictions applicable to functions.

19.4 No dynamic SQL using `PREPARE` and `EXECUTE`

Stored functions cannot use dynamic SQL statements such as `PREPARE`, `EXECUTE`, and `DEALLOCATE PREPARE` to execute dynamically constructed statements.

19.5 Restrictions on modifying tables

MySQL imposes restrictions on modifying a table that is already being used by the statement invoking the function.

For example, a function invoked from an `UPDATE` should not attempt to update the same table in a conflicting way.

A function is not a substitute for arbitrary data-modification workflows.

19.6 No `OUT` or `INOUT` parameters

Stored functions use input parameters. If you need `OUT` or `INOUT` parameters, consider a stored procedure.

19.7 No `RETURN` of a result set

A stored function returns a scalar value, not a table-valued result set.

If your requirement is to return several rows and columns, consider a normal `SELECT`, a view, or a stored procedure that returns a result set.

19.8 Other restrictions

MySQL also imposes restrictions involving certain statements, temporary tables, recursive stored-program calls, and other operations. The exact permitted behavior depends on the statement and context.

When writing production functions, consult the MySQL 8.0 reference manual's Stored Program Restrictions section rather than assuming that any SQL statement valid at the top level is valid inside a function.

20. Creating, Viewing, and Deleting Functions

20.1 List stored functions in the current database

```
SHOW FUNCTION STATUS
WHERE Db = DATABASE();
```

This displays metadata for stored functions in the current database.

20.2 View the definition

```
SHOW CREATE FUNCTION calculate_bonus;
```

This returns the stored definition, including its characteristics.

20.3 Query `INFORMATION_SCHEMA.ROUTINES`

```
SELECT
    ROUTINE_SCHEMA,
    ROUTINE_NAME,
    ROUTINE_TYPE,
    DATA_TYPE,
    IS_DETERMINISTIC,
    SQL_DATA_ACCESS
FROM INFORMATION_SCHEMA.ROUTINES
WHERE ROUTINE_SCHEMA = DATABASE()
  AND ROUTINE_TYPE = 'FUNCTION';
```

This is useful when inspecting multiple routines.

20.4 Drop a function

```
DROP FUNCTION calculate_bonus;
```

Safer form:

```
DROP FUNCTION IF EXISTS calculate_bonus;
```

`DROP FUNCTION` removes the stored function definition. It does not merely clear its result or reset its internal variables.

20.5 Modify an existing function

MySQL supports `ALTER FUNCTION` for changing certain characteristics, but it does not let you replace the body arbitrarily through a general-purpose `ALTER FUNCTION` statement.

For a body change, commonly use:

```
DROP FUNCTION IF EXISTS calculate_bonus;
```

Then recreate the function using `CREATE FUNCTION`.

Be careful: dropping and recreating a function can affect dependent objects, application code, permissions, and deployment workflows.

20.6 `CREATE FUNCTION IF NOT EXISTS`

MySQL 8.0.29 and later supports `CREATE FUNCTION IF NOT EXISTS` for stored functions.

Example:

```
CREATE FUNCTION IF NOT EXISTS simple_add(
    p_a INT,
    p_b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
RETURN p_a + p_b;
```

This does not replace an existing definition. If you need to update the function body, use an appropriate deployment strategy.

For compatibility with earlier MySQL versions, check the exact server version before using this syntax.

21. Function vs Stored Procedure

This is one of the most important interview topics.

Feature	Stored function	Stored procedure
Main purpose	Compute and return a value	Execute a workflow or operation
Definition	`CREATE FUNCTION`	`CREATE PROCEDURE`
Return declaration	Required using `RETURNS`	No function-style `RETURNS` declaration
Result	One scalar return value	Can return result sets and use output parameters
Parameters	Input parameters	`IN`, `OUT`, `INOUT`
Invocation	Commonly inside `SELECT` expressions	`CALL procedure_name(...)`
Can be used in `WHERE`	Yes, where the expression is valid	No
`RETURN`	Returns the function value	Exits the procedure; does not return a scalar value
Transaction control	Not permitted	Subject to procedure and transaction rules
Dynamic SQL	Not permitted	Can be used where permitted

Feature	Stored function	Stored procedure
Data modification	Subject to strict restrictions	Can perform permitted data modifications

Function example

```
DELIMITER //

CREATE FUNCTION add_two_numbers(
    p_a INT,
    p_b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_a + p_b;
END //

DELIMITER ;
```

Call:

```
SELECT add_two_numbers(10, 20);
```

Output: `30`

Procedure example

```
DELIMITER //

CREATE PROCEDURE show_sum(
    IN p_a INT,
    IN p_b INT
)
BEGIN
    SELECT p_a + p_b AS result;
END //

DELIMITER ;
```

Call:

```
CALL show_sum(10, 20);
```

Output:

result
30

Which should you use?

Use a function when you need a reusable scalar result in an expression.

Use a procedure when you need to perform a sequence of operations, return a result set, or use output parameters.

22. Function vs Trigger

Feature	Function	Trigger
Created using	`CREATE FUNCTION`	`CREATE TRIGGER`

Feature	Function	Trigger
Execution	Explicitly invoked by a query or expression	Automatically activated by a triggering table event
Parameters	Input parameters	No ordinary input parameter list
Return value	Scalar value	No scalar function-style return value
Main purpose	Reusable calculation or lookup	React to table changes
Can be called directly using `SELECT`	Yes	No

Example function call:

```
SELECT calculate_bonus(50000);
```

A trigger, by contrast, runs automatically when a configured event occurs, such as an insert or update on its associated table.

A trigger cannot be used as a replacement for a scalar function.

23. Important Rules and Common Mistakes

Mistake 1: Forgetting `RETURNS`

Incorrect:

```
CREATE FUNCTION add_numbers(p_a INT, p_b INT)
RETURN p_a + p_b;
```

Correct:

```
CREATE FUNCTION add_numbers(p_a INT, p_b INT)
RETURNS INT
DETERMINISTIC
NO SQL
RETURN p_a + p_b;
```

Mistake 2: Confusing `RETURN` and `RETURNS`

- `RETURNS INT` declares the type.
- `RETURN p_a + p_b` returns the value.

Mistake 3: Using `CALL` to invoke a function

Incorrect:

```
CALL add_numbers(10, 20);
```

Correct:

```
SELECT add_numbers(10, 20);
```

`CALL` is for stored procedures.

Mistake 4: Forgetting `DELIMITER`

In a command-line client or script runner that recognizes `DELIMITER`, omitting it around a compound definition can cause the body to be split at its internal semicolons.

Mistake 5: Declaring variables after executable statements

Incorrect:

```
BEGIN
    SET v_total = 100;
    DECLARE v_total INT;
    RETURN v_total;
END
```

Declare local variables at the beginning of the relevant block.

Mistake 6: Not handling NULL

A comparison such as `p_number = 0` does not detect `NULL`.

Use:

```
p_number IS NULL
```

when you need to detect missing input.

Mistake 7: Using the wrong return type

If the result can contain decimal places, a suitable `DECIMAL` type is usually more appropriate than `INT`.

Mistake 8: Returning an incorrect grade

Incorrect logic:

```
IF p_marks >= 50 THEN
    RETURN 'Pass';
ELSEIF p_marks >= 90 THEN
    RETURN 'A+';
END IF;
```

The `ELSEIF p_marks >= 90` branch can never be reached because any score of 90 or more already satisfies `p_marks >= 50`.

Check the more specific conditions first.

Mistake 9: Infinite loop

Every loop must eventually terminate, and loop variables must be updated correctly.

Mistake 10: Incorrect characteristic declarations

Do not claim `NO SQL` if the function reads from a table. Use an accurate data access characteristic.

Mistake 11: Assuming a function runs only once

When a function is used in a query over many rows, MySQL may evaluate it multiple times. Do not assume that repeated calls are cached.

Mistake 12: Assuming `DETERMINISTIC` means faster

`DETERMINISTIC` describes behavior; it is not a command to cache the function result.

24. Performance and Best Practices

1. Keep functions focused on one task.
2. Prefer simple expressions over loops when a formula can solve the problem.
3. Choose precise and appropriate return types.
4. Validate invalid input explicitly.
5. Handle `NULL` deliberately.

6. Use accurate function characteristics.
7. Avoid unnecessary table lookups inside functions called for many rows.
8. Check query execution plans when functions appear in filters or sorting.
9. Avoid expensive repeated calculations where a direct query or precomputed value would be more efficient.
10. Give functions descriptive names and document their business rules.
11. Use version-controlled scripts for function definitions.
12. Test boundary cases, missing values, and unexpected inputs.
13. Avoid hidden dependencies on mutable table data when deterministic behavior is required.
14. Keep permissions and binary logging requirements in mind.
15. Verify the behavior against the MySQL version used in your environment.

For a simple calculation, a function can improve readability. However, an equivalent direct SQL expression may be faster or easier for the optimizer to process in some queries.

25. Interview Questions and Answers

Q1. What is a user-defined function in MySQL?

A stored function is a named database object that accepts input parameters, executes logic, and returns one scalar value. It can be invoked inside SQL expressions where permitted.

Q2. Why do we use functions?

To encapsulate reusable logic, centralize calculations, improve readability, and maintain consistent business rules.

Q3. What is the difference between `RETURNS` and `RETURN`?

`RETURNS` declares the return data type in the definition. `RETURN` supplies the actual result during execution.

Q4. Can a MySQL function accept multiple parameters?

Yes. For example:

```
DELIMITER //

CREATE FUNCTION calculate_area(
    p_length DECIMAL(10,2),
    p_width DECIMAL(10,2)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC
NO SQL
RETURN p_length * p_width //

DELIMITER ;
```

```
SELECT calculate_area(10, 5);
```

Output: `50.00`.

Q5. Can a function return multiple values?

Not through the ordinary scalar return mechanism. It returns one value. A structured string or JSON value can contain multiple pieces of information, but that is still one returned value. Use procedures, result sets, or other database designs when multiple output columns or rows are required.

Q6. Can a function return `NULL`?

Yes, if the function's logic returns `NULL` or the returned expression evaluates to `NULL`. The return type must be compatible with that result.

Q7. Can we use a function inside `WHERE`?

Yes, where the expression is valid:

```
SELECT *
FROM employees
WHERE annual_salary(salary) > 600000;
```

However, performance depends on the expression and execution plan.

Q8. Can we use `SELECT` inside a function?

Yes, if the query is permitted and assigns its result appropriately, such as with `SELECT ... INTO`. A function cannot use an ordinary result-set-producing `SELECT` to send rows directly to the client.

Q9. Can a function modify a table?

Stored functions have strict restrictions on data-modification statements. In particular, they cannot perform unrestricted updates or inserts as a general-purpose workflow, and MySQL prohibits certain modifications to tables used by the invoking statement. Use a stored procedure or application transaction when appropriate.

Q10. What is a deterministic function?

A deterministic function returns the same result for the same inputs under the same relevant conditions.

Example: `square_value(5)` returns `25`.

Q11. What is a non-deterministic function?

A non-deterministic function may return different results for the same inputs.

Example: a function returning the current timestamp.

Q12. Can we use `OUT` parameters in a function?

No. Stored functions use input parameters. Stored procedures support `OUT` and `INOUT`.

Q13. What is the purpose of `DELIMITER`?

It changes the client-side statement terminator so semicolons inside the function body do not prematurely end the `CREATE FUNCTION` statement.

Q14. Can a function contain an `IF` statement?

Yes. Stored functions support `IF ... THEN ... ELSEIF ... ELSE ... END IF`.

Q15. Can a function contain loops?

Yes. `WHILE`, `REPEAT`, and `LOOP` are supported. The loop must be designed to terminate correctly.

Q16. What is the difference between a function and a procedure?

A function returns a scalar value and can be used inside expressions. A procedure is invoked using `CALL` and can perform workflows, return result sets, and use output parameters.

Q17. What happens if `SELECT ... INTO` returns no rows?

A `NOT FOUND` condition occurs. Without suitable handling, the function may fail. A condition handler can define the desired behavior.

Q18. Can we drop a function?

Yes:

```
DROP FUNCTION IF EXISTS calculate_bonus;
```

Q19. How do you inspect a function's definition?

```
SHOW CREATE FUNCTION calculate_bonus;
```

Q20. Does `DETERMINISTIC` cache function results?

No. It describes expected behavior; it does not guarantee caching or memoization.

26. Practice Questions with Solutions

Try solving each question before reading its answer.

Practice 1: Square a number

Question: Create `square_number(p_number)` to return the square.

```
DELIMITER //

CREATE FUNCTION square_number(
    p_number INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
RETURN p_number * p_number //

DELIMITER ;
```

Test:

```
SELECT square_number(7);
```

Output: `49`.

Practice 2: Calculate cube

Question: Return the cube of a number.

```
DELIMITER //

CREATE FUNCTION cube_number(
    p_number INT
)
RETURNS BIGINT
DETERMINISTIC
NO SQL
RETURN p_number * p_number * p_number //

DELIMITER ;
```

Test:

```
SELECT cube_number(3);
```

Output: `27`.

Practice 3: Find the larger number

```
DELIMITER //

CREATE FUNCTION larger_number(
    p_a INT,
    p_b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    IF p_a IS NULL OR p_b IS NULL THEN
        RETURN NULL;
    ELSEIF p_a > p_b THEN
        RETURN p_a;
    ELSE
        RETURN p_b;
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT larger_number(25, 40);
```

Output: `40`.

Practice 4: Calculate employee experience bonus

Business rule: 1,000 per completed year of experience.

```
DELIMITER //

CREATE FUNCTION experience_bonus(
    p_years INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_years IS NULL OR p_years < 0 THEN
        RETURN NULL;
    END IF;

    RETURN p_years * 1000.00;
END //

DELIMITER ;
```

Test:

```
SELECT experience_bonus(5);
```

Output: `5000.00`.

Practice 5: Determine pass or fail

```
DELIMITER //

CREATE FUNCTION pass_fail(
    p_marks DECIMAL(5,2)
)
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
    IF p_marks IS NULL OR p_marks < 0 OR p_marks > 100 THEN
        RETURN 'Invalid';
    ELSEIF p_marks >= 40 THEN
        RETURN 'Pass';
    ELSE
        RETURN 'Fail';
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT pass_fail(75);
SELECT pass_fail(25);
SELECT pass_fail(120);
```

Expected: `Pass`, `Fail`, `Invalid`.

Practice 6: Return department employee count

Question: Create a function that accepts a department name and returns the number of employees in that department.

```
DELIMITER //

CREATE FUNCTION department_count(
    p_department VARCHAR(50)
)
RETURNS BIGINT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count BIGINT;

    SELECT COUNT(*)
    INTO v_count
    FROM employees
    WHERE department = p_department;

    RETURN v_count;
END //

DELIMITER ;
```

Test:

```
SELECT department_count('IT');
```

Output: `3`.

Logic: The function counts matching employee rows and returns the count. If the department has no matching rows, `COUNT(*)` returns `0`.

Practice 7: Calculate total salary for a department

```
DELIMITER //

CREATE FUNCTION department_salary(
    p_department VARCHAR(50)
)
RETURNS DECIMAL(14,2)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(14,2);

    SELECT COALESCE(SUM(salary), 0)
    INTO v_total
    FROM employees
    WHERE department = p_department;

    RETURN v_total;
END //

DELIMITER ;
```

Test:

```
SELECT department_salary('IT');
```

Output: `145000.00`.

Logic: The IT employees earn 50,000 + 65,000 + 30,000 = 145,000.

Practice 8: Return the highest salary

```
DELIMITER //

CREATE FUNCTION highest_salary()
RETURNS DECIMAL(10,2)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_salary DECIMAL(10,2);

    SELECT MAX(salary)
    INTO v_salary
    FROM employees;

    RETURN v_salary;
END //

DELIMITER ;
```

Test:

```
SELECT highest_salary();
```

Output: `65000.00`.

Practice 9: Return salary category

Rules:

- Below 35,000: `Low`
- 35,000 to below 55,000: `Medium`
- 55,000 or above: `High`

```
DELIMITER //

CREATE FUNCTION salary_category(
    p_salary DECIMAL(10,2)
)
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
    IF p_salary IS NULL OR p_salary < 0 THEN
        RETURN 'Invalid';
    ELSEIF p_salary < 35000 THEN
        RETURN 'Low';
    ELSEIF p_salary < 55000 THEN
        RETURN 'Medium';
    ELSE
        RETURN 'High';
    END IF;
END //

DELIMITER ;
```

Test:

```
SELECT salary_category(30000);
SELECT salary_category(50000);
SELECT salary_category(65000);
```

Expected: `Low`, `Medium`, `High`.

Practice 10: Count numbers divisible by 3

```
DELIMITER //

CREATE FUNCTION count_divisible_by_3(
    p_n INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_i INT DEFAULT 1;
    DECLARE v_count INT DEFAULT 0;

    IF p_n IS NULL OR p_n <= 0 THEN
        RETURN 0;
    END IF;

    WHILE v_i <= p_n DO
        IF MOD(v_i, 3) = 0 THEN
            SET v_count = v_count + 1;
        END IF;

        SET v_i = v_i + 1;
    END WHILE;

    RETURN v_count;
END //

DELIMITER ;
```

Test:

```
SELECT count_divisible_by_3(10);
```

Output: `3`.

The numbers divisible by 3 from 1 to 10 are 3, 6, and 9.

27. Quick Revision Sheet

Concept	Remember
Create function	`CREATE FUNCTION`
Input parameters	Parameter name and data type
Return type	`RETURNS data_type`
Return value	`RETURN expression`
Compound body	`BEGIN ... END`
Local variable	`DECLARE`
Assign variable	`SET`
Query into variable	`SELECT ... INTO`
Conditional	`IF`, `CASE`
Loops	`WHILE`, `REPEAT`, `LOOP`
Exit loop	`LEAVE`
Skip iteration	`ITERATE`
Determinism	`DETERMINISTIC`, `NOT DETERMINISTIC`
Data access	`NO SQL`, `CONTAINS SQL`, `READS SQL DATA`, `MODIFIES SQL DATA`
Invoke function	`SELECT function_name(...)`
Inspect definition	`SHOW CREATE FUNCTION`
Inspect metadata	`INFORMATION_SCHEMA.ROUTINES`
Delete function	`DROP FUNCTION IF EXISTS`

Concept	Remember
Procedure invocation	`CALL procedure_name(...)`

Final mental model

Think of a stored function as a reusable calculation box:

Input parameters

Example: salary = 50,000

Function logic

Declare variables, calculate, validate, apply conditions

RETURN

Example: bonus = 5,000

Calling SQL statement

SELECT calculate_bonus(50000);

The most important distinction to remember is this: a stored function is designed to return a scalar value that can participate in an SQL expression, whereas a stored procedure is designed to execute a broader operation or workflow.

For the official MySQL 8.0 documentation, the key reference sections are CREATE PROCEDURE and CREATE FUNCTION Syntax, Stored Program Restrictions, and Stored Object Access Control.